

CS5481 - Tutorial 3

Data Preprocessing and Regularization Expression

Welcome to CS5481 tutorial. In this tutorial, you will learn to how to preprocess data you collected with Pandas and be familiar with Regular Expression and utilize it to process text data.

Preparation

- Python
- Python Libraries
 - Pandas
 - re

```
In [ ]: !pip install pandas
```

1. Import Libraries

```
In [1]: import pandas as pd
```

2. Data Preprocessing

2.1 Data Cleaning

- Basic Operations
- Check NAN Data
- Check Unreasonable Data
- Check Replicated Data
- Constrain Data Type
- Save Files

```
In [2]: data = pd.read_csv('movie_metadata.csv', encoding="utf-8")
```

- Basic Operations

```
In [3]: # show the first 5 lines of the file  
data.head()
```

Out[3]: Columns 1–8 of 22

	color	director_name	num_critics_for_reviews	duration	director_facebook_likes	actor_3_facebook_likes	actor_2_name
0	Color	James Cameron	723.0	178.0	0.0	855.0	Joel David Moore
1	Color	Gore Verbinski	302.0	169.0	563.0	1000.0	Orlando Bloom
2	Color	Sam Mendes	602.0	148.0	0.0	161.0	Rory Kinnear
3	Color	Christopher Nolan	813.0	164.0	22000.0	23000.0	Christian Bale
4	NaN	Doug Walker	NaN	NaN	131.0	NaN	Rob Walker

Columns 9–15 of 22 · first column repeated

	actor_1_facebook_likes	gross	genres	...	num_user_for_reviews	language	country
0	1000.0	760505847.0	Action Adventure Fantasy Sci-Fi	...	3054.0	English	USA
1	40000.0	309404152.0	Action Adventure Fantasy	...	1238.0	English	USA
2	11000.0	200074175.0	Action Adventure Thriller	...	994.0	English	UK
3	27000.0	448130642.0	Action Thriller	...	2701.0	English	USA
4	131.0	NaN	Documentary	...	NaN	NaN	NaN

Columns 16–22 of 22 · first column repeated

	content_rating	budget	title_year	actor_2_facebook_likes	imdb_score	aspect_ratio	movie_facebook_likes
0	PG-13	237000000.0	2009.0	936.0	7.9	1.78	33000
1	PG-13	300000000.0	2007.0	5000.0	7.1	2.35	0
2	PG-13	245000000.0	2015.0	393.0	6.8	2.35	85000
3	PG-13	250000000.0	2012.0	23000.0	8.5	2.35	164000
4	NaN	NaN	NaN	12.0	7.1	NaN	0

5 rows × 28 columns

In [5]: `# show the last 5 lines of the file`
`data.tail()`

Out[5]: Columns 1–8 of 22

	color	director_name	num_critics_for_reviews	duration	director_facebook_likes	actor_3_facebook_likes	actor_2_name
5038	Color	Scott Smith	1.0	87.0	2.0	318.0	Daphne Zuniga
5039	Color	NaN	43.0	43.0	NaN	319.0	Valorie Curry
5040	Color	Benjamin Roberds	13.0	76.0	0.0	0.0	Maxwell Moody
5041	Color	Daniel Hsia	14.0	100.0	0.0	489.0	Daniel Henney
5042	Color	Jon Gunn	43.0	90.0	16.0	16.0	Brian Herzlinger

Columns 9–15 of 22 · first column repeated

	actor_1_facebook_likes	gross	genres	...	num_user_for_reviews	language	country
5038	637.0	NaN	Comedy Drama	...	6.0	English	Canada
5039	841.0	NaN	Crime Drama Mystery Thriller	...	359.0	English	USA
5040	0.0	NaN	Drama Horror Thriller	...	3.0	English	USA
5041	946.0	10443.0	Comedy Drama Romance	...	9.0	English	USA
5042	86.0	85222.0	Documentary	...	84.0	English	USA

Columns 16–22 of 22 · first column repeated

	content_rating	budget	title_year	actor_2_facebook_likes	imdb_score	aspect_ratio	movie_facebook_likes
5038	NaN	NaN	2013.0	470.0	7.7	NaN	84
5039	TV-14	NaN	NaN	593.0	7.5	16.00	32000
5040	NaN	1400.0	2013.0	0.0	6.3	NaN	16
5041	PG-13	NaN	2012.0	719.0	6.3	2.35	660
5042	PG	1100.0	2004.0	23.0	6.6	1.85	456

5 rows × 28 columns

```
In [4]: # check stat info of columns: data.columnname.describe()  
data.duration.describe()
```

```
Out[4]: count    5028.000000  
mean      107.201074  
std       25.197441  
min        7.000000  
25%       93.000000  
50%      103.000000  
75%      118.000000  
max      511.000000  
Name: duration, dtype: float64
```

```
In [6]: # choose a column: data[columnname]  
data['color']
```

```
Out[6]: 0      Color  
1      Color  
2      Color  
3      Color  
4      NaN  
  
...  
5038    Color  
5039    Color  
5040    Color  
5041    Color  
5042    Color  
Name: color, Length: 5043, dtype: object
```

```
In [7]: # choose the first K lines: data['columnname'][:K]  
K = 10  
data["color"][:K]
```

```
Out[7]: 0      Color  
1      Color  
2      Color  
3      Color  
4      NaN  
5      Color  
6      Color  
7      Color  
8      Color  
9      Color  
Name: color, dtype: object
```

```
In [8]: # choose multiple columns: data[["column1", "column2"]]  
data[["color", "director_name"]]
```

Out[8]:

	color	director_name
0	Color	James Cameron
1	Color	Gore Verbinski
2	Color	Sam Mendes
3	Color	Christopher Nolan
4	NaN	Doug Walker
...	...	...
5038	Color	Scott Smith
5039	Color	NaN
5040	Color	Benjamin Roberds
5041	Color	Daniel Hsia
5042	Color	Jon Gunn

5043 rows × 2 columns

```
In [9]: # where filtering: data[data['columnname'] > condition]  
# choose films whose duration is larger than 150 mins  
data[data["duration"] > 150]
```

Out[9]: Columns 1–8 of 22

	color	director_name	num_critics_for_reviews	duration	director_facebook_likes	actor_3_facebook_likes	actor_2_name
0	Color	James Cameron	723.0	178.0	0.0	855.0	Joel David Moore
1	Color	Gore Verbinski	302.0	169.0	563.0	1000.0	Orlando Bloom
3	Color	Christopher Nolan	813.0	164.0	22000.0	23000.0	Christian Bale
6	Color	Sam Raimi	392.0	156.0	0.0	4000.0	James Franco
9	Color	David Yates	375.0	153.0	282.0	10000.0	Daniel Radcliffe
...	...	...	...	...	...	...	...
4688	Color	Steve James	53.0	170.0	23.0	2.0	Arthur Agee
4694	Color	Peter Jackson	446.0	201.0	0.0	84.0	Thomas Kretschmann
4708	Color	Michael Wadleigh	53.0	215.0	14.0	136.0	Jimi Hendrix
4747	Black and White	Akira Kurosawa	153.0	202.0	0.0	4.0	Minoru Chiaki
4885	Black and White	King Vidor	48.0	151.0	54.0	6.0	Renée Adorée

Columns 9–15 of 22 · first column repeated

	actor_1_facebook_likes	gross	genres	...	num_user_for_reviews	language	country
0	1000.0	760505847.0	Action Adventure Fantasy Sci-Fi	...	3054.0	English	USA
1	40000.0	309404152.0	Action Adventure Fantasy	...	1238.0	English	USA
3	27000.0	448130642.0	Action Thriller	...	2701.0	English	USA
6	24000.0	336530303.0	Action Adventure Romance	...	1902.0	English	USA
9	25000.0	301956980.0	Adventure Family Fantasy Mystery	...	973.0	English	UK
...	...	...	...	...	...	...	...

	actor_1_fac ebook_likes	gross	genres	...	num_user_f or_reviews	language	country
4688	7.0	7830611.0	Documenta ry Drama S port	...	74.0	English	USA
4694	6000.0	218051260. 0	Action Adv enture Dra ma Romanc e	...	2618.0	English	New Zealand
4708	262.0	13300000.0	Documenta ry History Music	...	63.0	English	USA
4747	304.0	269061.0	Action Adv enture Dra ma	...	596.0	Japanese	Japan
4885	81.0	NaN	Drama Rom ance War	...	45.0	NaN	USA

Columns 16–22 of 22 · first column repeated

	content_rat ing	budget	title_year	actor_2_fac ebook_likes	imdb_score	aspect_rati o	movie_face book_likes
0	PG-13	237000000. .0	2009.0	936.0	7.9	1.78	33000
1	PG-13	300000000. .0	2007.0	5000.0	7.1	2.35	0
3	PG-13	250000000. .0	2012.0	23000.0	8.5	2.35	164000
6	PG-13	258000000. .0	2007.0	11000.0	6.2	2.35	0
9	PG	250000000. .0	2009.0	11000.0	7.5	2.35	10000
...	...	...	...	...	...	...	...
4688	PG-13	700000.0	1994.0	6.0	8.3	1.33	0
4694	PG-13	207000000. .0	2005.0	918.0	7.2	2.35	0
4708	R	600000.0	1970.0	227.0	8.1	2.20	0
4747	Unrated	2000000.0	1954.0	8.0	8.7	1.37	11000
4885	Not Rated	245000.0	1925.0	12.0	8.3	1.33	226

205 rows × 28 columns

- Process NAN Data

1. fill value
2. remove corresponding lines
3. remove columns where many values are nan

```
In [10]: # check all nan data  
data.isna()
```

Out[10]: Columns 1–8 of 22

	color	director_name	num_critics_for_reviews	duration	director_facebook_likes	actor_3_facebook_likes	actor_2_name
0	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False
4	True	False	True	True	False	True	False
...	...	...	...	...	...	...	...
5038	False	False	False	False	False	False	False
5039	False	True	False	False	True	False	False
5040	False	False	False	False	False	False	False
5041	False	False	False	False	False	False	False
5042	False	False	False	False	False	False	False

Columns 9–15 of 22 · first column repeated

	actor_1_facebook_likes	gross	genres	...	num_user_for_reviews	language	country
0	False	False	False	...	False	False	False
1	False	False	False	...	False	False	False
2	False	False	False	...	False	False	False
3	False	False	False	...	False	False	False
4	False	True	False	...	True	True	True
...	...	...	...	...	...	...	...
5038	False	True	False	...	False	False	False
5039	False	True	False	...	False	False	False
5040	False	True	False	...	False	False	False
5041	False	False	False	...	False	False	False
5042	False	False	False	...	False	False	False

Columns 16–22 of 22 · first column repeated

	content_rating	budget	title_year	actor_2_facebook_likes	imdb_score	aspect_ratio	movie_facebook_likes
0	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False
4	True	True	True	False	False	True	False
...	...	...	...	...	...	...	...
5038	True	True	False	False	False	True	False

	content_rating	budget	title_year	actor_2_facebook_likes	imdb_score	aspect_ratio	movie_facebook_likes
5039	False	True	True	False	False	False	False
5040	True	False	False	False	False	True	False
5041	False	True	False	False	False	False	False
5042	False	False	False	False	False	False	False

5043 rows × 28 columns

```
In [11]: # fill data with suitable values
# for example, use "cs5481" to replace nan values in column "country"
data.country = data.country.fillna("cs5481")
print(data.head().country)
# use mean duration to replace nan values in column "duration"
data.duration = data.duration.fillna(data.duration.mean())
print(data.duration)
```

```
0      USA
1      USA
2      UK
3      USA
4  cs5481
Name: country, dtype: object
0      178.000000
1      169.000000
2      148.000000
3      164.000000
4      107.201074
...
5038    87.000000
5039    43.000000
5040    76.000000
5041   100.000000
5042    90.000000
Name: duration, Length: 5043, dtype: float64
```

```
In [12]: # remove lines where some values are nan  
data.dropna()
```

Out[12]: Columns 1–8 of 22

	color	director_name	num_critics_for_reviews	duration	director_facebook_likes	actor_3_facebook_likes	actor_2_name
0	Color	James Cameron	723.0	178.0	0.0	855.0	Joel David Moore
1	Color	Gore Verbinski	302.0	169.0	563.0	1000.0	Orlando Bloom
2	Color	Sam Mendes	602.0	148.0	0.0	161.0	Rory Kinnear
3	Color	Christopher Nolan	813.0	164.0	22000.0	23000.0	Christian Bale
5	Color	Andrew Stanton	462.0	132.0	475.0	530.0	Samantha Morton
...	...	...	...	...	...	...	...
5026	Color	Olivier Assayas	81.0	110.0	107.0	45.0	Béatrice Dalle
5027	Color	Jafar Panahi	64.0	90.0	397.0	0.0	Nargess Mamizadeh
5033	Color	Shane Carruth	143.0	77.0	291.0	8.0	David Sullivan
5035	Color	Robert Rodriguez	56.0	81.0	0.0	6.0	Peter Marquardt
5042	Color	Jon Gunn	43.0	90.0	16.0	16.0	Brian Herzlinger

Columns 9–15 of 22 · first column repeated

	actor_1_facebook_likes	gross	genres	...	num_user_for_reviews	language	country
0	1000.0	760505847.0	Action Adventure Fantasy Sci-Fi	...	3054.0	English	USA
1	40000.0	309404152.0	Action Adventure Fantasy	...	1238.0	English	USA
2	11000.0	200074175.0	Action Adventure Thriller	...	994.0	English	UK
3	27000.0	448130642.0	Action Thriller	...	2701.0	English	USA
5	640.0	73058679.0	Action Adventure Sci-Fi	...	738.0	English	USA
...	...	...	...	...	...	...	...
5026	576.0	136007.0	Drama Music Romance	...	39.0	French	France
5027	5.0	673780.0	Drama	...	26.0	Persian	Iran

	actor_1_fac ebook_likes	gross	genres	...	num_user_f or_reviews	language	country
5033	291.0	424760.0	Drama Sci-Fi Thriller	...	371.0	English	USA
5035	121.0	2040920.0	Action Crime Drama Romance Thriller	...	130.0	Spanish	USA
5042	86.0	85222.0	Documentary	...	84.0	English	USA

Columns 16–22 of 22 · first column repeated

	content_rating	budget	title_year	actor_2_fac ebook_likes	imdb_score	aspect_ratio	movie_facebook_likes
0	PG-13	237000000.0	2009.0	936.0	7.9	1.78	33000
1	PG-13	300000000.0	2007.0	5000.0	7.1	2.35	0
2	PG-13	245000000.0	2015.0	393.0	6.8	2.35	85000
3	PG-13	250000000.0	2012.0	23000.0	8.5	2.35	164000
5	PG-13	263700000.0	2012.0	632.0	6.6	2.35	24000
...	...	...	...	...	...	...	...
5026	R	4500.0	2004.0	133.0	6.9	2.35	171
5027	Not Rated	10000.0	2000.0	0.0	7.5	1.85	697
5033	PG-13	7000.0	2004.0	45.0	7.0	1.85	19000
5035	R	7000.0	1992.0	20.0	6.9	1.37	0
5042	PG	1100.0	2004.0	23.0	6.6	1.85	456

3755 rows × 28 columns

```
In [ ]: # dropping lines where just some values are nan is aggressive, so we can just
        # remove lines where all values are nan.
        data.dropna(how="all")
```

```
In [ ]: # we can also add some limitations, save lines where more than 25 values are
        # not nan
        data.dropna(thresh=25)
```

```
In [ ]: # we can remove columns where all values are nan
        data.dropna(axis=1, how="all")
```

```
In [ ]: # or remove columns where some values are nan
        data.dropna(axis=1, how="any")
```

- Check Unreasonable Data

1. Time

2. Values with a range

```
In [ ]: # check title_year
data[data["title_year"] > 2015]
```

```
In [ ]: # check imdb_score
data[data["imdb_score"] > 10]
```

- Check Replicated Data

```
In [ ]: # check duplicated data
data.duplicated()
```

```
In [ ]: # There is no duplicated lines in this file, thus we use a demo data to show
        # how process it.
df = pd.DataFrame({
    'brand': ['Yum Yum', 'Yum Yum', 'Indomie', 'Indomie', 'Indomie'],
    'style': ['cup', 'cup', 'cup', 'pack', 'pack'],
    'rating': [4, 4, 3.5, 15, 5]
})
df
```

```
In [ ]: # By default, for each set of duplicated values, the first occurrence is set
        # on False and all others on True.
df.duplicated()
```

```
In [ ]: # By using 'last', the last occurrence of each set of duplicated values is
        # set on False and all others on True.
df.duplicated(keep='last')
```

```
In [ ]: # By setting keep on False, all duplicates are True.
df.duplicated(keep=False)
```

```
In [ ]: # To find duplicates on specific column(s), use subset.
df.duplicated(subset=['brand'])
```

- Constrain Data Type

```
In [ ]: # we can assume we know some columns' types and we can predefine it when
        # reading data
data = pd.read_csv(r'movie_metadata.csv', dtype={'num_voted_users': int,
        "title_year": str})
```

```
In [ ]: # we can also rename columns for human understanding
data = data.rename(columns = {'title_year':'release_date',
                              'movie_facebook_likes':'facebook_likes'})
data.head()
```

```
In [ ]: # after clean the data, we usually need to save the cleaned data to a new
file
data.to_csv('cleanfile.csv', encoding='utf-8')
```

2.2 Data Integration

```
In [ ]: df1=pd.DataFrame({'key':['b','b','a','c','a','a','b'],'data1':range(7)})
df1
```

```
In [ ]: df2=pd.DataFrame({'key':['a','b','d'],'data2':range(3)})
df2
```

```
In [ ]: # we can merge two datasets with pd.merge(), the default merged column is
the common column
pd.merge(df1, df2)
```

```
In [ ]: # of course, we could give the merged column
pd.merge(df1,df2,on='key')
```

```
In [ ]: # we could merge two datasets with different columns
df3=pd.DataFrame({'1key':['b','b','a','c','a','a','b'],'data1':range(7)})
df3
```

```
In [ ]: df4=pd.DataFrame({'2key':['a','b','d'],'data2':range(3)})
df4
```

```
In [ ]: # default mode keeps the cross set of key values, which is called inner
connection
pd.merge(df3,df4,left_on='1key',right_on='2key')
```

```
In [ ]: # when merge two datasets with outer connection
print(df1)
print(df2)
pd.merge(df1,df2,on='key',how='outer')
```

2.3 Data Transformation

- String Transformation
- Number Transformation
- String Transformation

```
In [ ]: # lower and upper case
data["director_name"].str.upper()
```

```
In [ ]: # lower and upper case
data["director_name"].str.lower()
```

```
In [ ]: # remove special strings for whitespace, \n
data['movie_title'].str.strip()
```

• Number Transformation

```
In [ ]: # unit transformation

data["duration"]
```

```
In [ ]: data["duration"] / 60
```

```
In [ ]: # normalization
norm_duration = (data.duration - data.duration.min()) / (data.duration.max()
- data.duration.min())
norm_duration
```

```
In [ ]: # standardization
std_duration = (data.duration - data.duration.mean()) / data.duration.std()
std_duration
```

```
In [ ]: # discretization
# qcut: divie data points into M groups and each group has the basically
same number of data points
m_cut = pd.qcut(data.duration, 5)
m_cut.value_counts()
```

3. Regular Expression

1. Metacharacters

- `[]` A set of characters
- `\` Signals a special sequence (can also be used to escape special characters)
- `.` Any character (except newline character) "he..o"
- `^` Starts with "^hello"
- Ends with "planet\$"
- `*` Zero or more occurrences "he.*o"
- `+` One or more occurrences "he.+o"
- `?` Zero or one occurrences "he.?o"
- `{}` Exactly the specified number of occurrences "he.{2}o"
- `|` Either or "falls|stays"

- `()` Capture and group

2.Special Sequences

- `\A` Returns a match if the specified characters are at the beginning of the string `"\AThe"`
- `\b` Returns a match where the specified characters are at the beginning or at the end of a word (the `"r"` in the beginning is making sure that the string is being treated as a "raw string")
`r"\bain" r"ain\b"`
- `\B` Returns a match where the specified characters are present, but NOT at the beginning (or at the end) of a word (the `"r"` in the beginning is making sure that the string is being treated as a "raw string") `r"\Bain" r"ain\B"`
- `\d` Returns a match where the string contains digits (numbers from 0-9) `"\d"`
- `\D` Returns a match where the string DOES NOT contain digits `"\D"`
- `\s` Returns a match where the string contains a white space character `"\s"`
- `\S` Returns a match where the string DOES NOT contain a white space character `"\S"`
- `\w` Returns a match where the string contains any word characters (characters from a to Z, digits from 0-9, and the underscore `_` character) `"\w"`
- `\W` Returns a match where the string DOES NOT contain any word characters `"\W"`
- `\Z` Returns a match if the specified characters are at the end of the string

3.Sets

- `[arn]` Returns a match where one of the specified characters (a, r, or n) is present
- `[a-n]` Returns a match for any lower case character, alphabetically between a and n
- `[^arn]` Returns a match for any character EXCEPT a, r, and n
- `[0123]` Returns a match where any of the specified digits (0, 1, 2, or 3) are present
- `[0-9]` Returns a match for any digit between 0 and 9
- `[0-5][0-9]` Returns a match for any two-digit numbers from 00 and 59
- `[a-zA-Z]` Returns a match for any character alphabetically between a and z, lower case OR upper case
- `[+]` In sets, `+`, `*`, `.`, `|`, `()`, `$`, `{}` has no special meaning, so `[+]` means: return a match for any `+` character in the string

4.Functions

- The `findall()` function returns a list containing all matches.
- The `search()` function searches the string for a match, and returns a `Match` object if there is a match. If there is more than one match, only the first occurrence of the match will be returned. If no matches are found, the value `None` is returned.
- The `split()` function returns a list where the string has been split at each match.
- The `sub()` function replaces the matches with the text of your choice.

5.Objects

- A `Match` Object is an object containing information about the search and the result.

- The Match object has properties and methods used to retrieve information about the search, and the result:
- `.span()` returns a tuple containing the start-, and end positions of the match.
- `.string` returns the string passed into the function
- `.group()` returns the part of the string where there was a match

```
In [ ]: import re
```

```
In [ ]: # findall function
txt = "The rain in Spain"
x = re.findall("ai", txt)
print(x)
```

```
In [ ]: # Return an empty list if no match was found:
txt = "The rain in Spain"
x = re.findall("Portugal", txt)
print(x)
```

```
In [ ]: # search function
txt = "The rain in Spain"
x = re.search("\s", txt)

print("The first white-space character is located in position:", x.start())
```

```
In [ ]: # If no matches are found, the value None is returned:
txt = "The rain in Spain"
x = re.search("Portugal", txt)
print(x)
```

```
In [ ]: # split function
txt = "The rain in Spain"
x = re.split("\s", txt)
print(x)
```

```
In [ ]: # Split the string only at the first occurrence:
txt = "The rain in Spain"
x = re.split("\s", txt, 1)
print(x)
```

```
In [ ]: # Replace every white-space character with the number 9:
txt = "The rain in Spain"
x = re.sub("\s", "9", txt)
print(x)
```

```
In [ ]: # Replace the first 2 occurrences:
txt = "The rain in Spain"
x = re.sub("\s", "9", txt, 2)
print(x)
```

```
In [ ]: # Do a search that will return a Match Object:
txt = "The rain in Spain"
x = re.search("ai", txt)
print(x) #this will print an object
```

```
In [ ]: # Print the position (start- and end-position) of the first match
occurrence.
# The regular expression looks for any words that starts with an upper case
"S":
txt = "The rain in Spain"
x = re.search(r"\bS\w+", txt)
print(x.span())
```

```
In [ ]: # Print the string passed into the function:
txt = "The rain in Spain"
x = re.search(r"\bS\w+", txt)
print(x.string)
```

```
In [ ]: # Print the part of the string where there was a match.
# The regular expression looks for any words that starts with an upper case
"S":
txt = "The rain in Spain"
x = re.search(r"\bS\w+", txt)
print(x.group())
```

```
In [ ]: # Construction a regular expression which could extract e-mail
pattern = re.compile(r"[a-zA-Z0-9_-]+@[a-zA-Z0-9_-]+(?:\.[a-zA-Z0-9_-]+)")

strs = 'My personal e-mail is test@outlook.com, company e-mail is
123456@qq.com'
result = pattern.findall(strs)

print(result)
```

4. Practice

data preprocessing

```
In [ ]: a = [[1, 2, 3, 4, ],
              [2, 3, 4, 5, 6],
              [3, 4, 5, 6, 7]]
data = pd.DataFrame(a)
data
```

```
In [ ]: # obtain the normalized data
        norm_data =
        norm_data
```

```
In [ ]: # obtain the standarized data
        std_data =
        std_data
```

regularization

```
In [ ]: # Construct a regular expression which could extract date
        pattern = #insert your answer here
        strs = 'Today is 2022/09/13, today in the last year is 2021.09.13, today in
the next year is 2023-09-13'
        result = #insert your answer here
        print(result)
        # The answer is
```